

Student Name	YAHAYA DANJUMA ADDRA	Registration No.	C113/26/OCO/17001
Programme/Cohort	OCO/Cohort11	Date	02/10/2026
Instructor	Aminu Idris AMCPN	Score	/100
Environment	Kali Linux / Ubuntu	Submission	E-Campus

LAB REPORT ON NETWORK AUTOMATION AND INTEGRATED SECURITY CHALLENGE

TASK 1 REPORT

RESPONSE:

In this Lab, I planned what each tool should do and the limits it should follow. The network inventory checks my computer, the ping sweep checks the approved network, the service checker checks selected ports, and the PCAP tool analyses an existing capture file. I also made sure that invalid or unauthorized inputs would be rejected so that the tools are used safely within the lab environment.

TASK 2 REPORT

EVIDENCE:

```

(kali@kali)-[~/Documents/ICDFA_LABS]
$ mkdir -p core102_lab5

(kali@kali)-[~/Documents/ICDFA_LABS]
$ cd core102_lab5

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ nano network_inventory.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ chmod +x network_inventory.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ bash -n network_inventory.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./network_inventory.sh
Inventory saved to evidence/network_inventory_20261002_105525.txt

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./network_inventory.sh custom_evidence
Inventory saved to custom_evidence/network_inventory_20261002_105609.txt

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ find . -maxdepth 2 -type f -name 'network_inventory_*.txt' -print
./evidence/network_inventory_20261002_105525.txt
./custom_evidence/network_inventory_20261002_105609.txt

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ sed -n '1,160p' evidence/network_inventory_*.txt

```

```
Session  Actions  Edit  View  Help
lo        UNKNOWN    00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0      UP          00:0c:29:f7:72:d2 <BROADCAST,MULTICAST,UP,LOWER_UP>
eth1      UP          00:0c:29:f7:72:dc <BROADCAST,MULTICAST,UP,LOWER_UP>
lo        UNKNOWN    127.0.0.1/8 ::1/128
eth0      UP          192.168.149.128/24 fe80::96d8:5426:4bfc:2254/64
eth1      UP          192.168.113.128/24 fe80::e475:42ea:1db0:f1f7/64

===== ROUTES =====
default via 192.168.149.2 dev eth0 proto dhcp src 192.168.149.128 metric 102
192.168.113.0/24 dev eth1 proto kernel scope link src 192.168.113.128 metric 101
192.168.149.0/24 dev eth0 proto kernel scope link src 192.168.149.128 metric 102

===== NEIGHBOURS =====
192.168.149.254 dev eth0 lladdr 00:50:56:ef:de:09 STALE
192.168.113.129 dev eth1 lladdr 00:0c:29:8c:19:cd STALE
192.168.113.254 dev eth1 lladdr 00:50:56:f4:25:ff STALE
192.168.149.2 dev eth0 lladdr 00:50:56:fc:63:12 STALE
192.168.113.1 dev eth1 lladdr 00:50:56:c0:00:01 STALE

===== LISTENING SOCKETS =====
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port

===== RESOLVER =====
# Generated by NetworkManager
search localdomain
nameserver 192.168.149.2
nameserver 192.168.113.1

===== DISK SPACE =====
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        79G   73G  1.6G  98% /
```

RESPONSE:

Set `-e` means stop on error and don't continue, `-u` detects missing variable, `pipefail` detects failure inside pipeline.

In this Lab I created a bash script that automatically inspect on my system's network including the interfaces, ip addresses, route table, ARP and whole lots. I learned that script helps to reduce manual process and makes inspection of system automatic.

TASK 3 REPORT

EVIDENCE:

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1       79G   73G   1.6G  98% /

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ nano authorised_ping_sweep.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ chmod +x authorised_ping_sweep.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ bash -n authorised_ping_sweep.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./authorised_ping_sweep.sh 203.0.113
zsh: no such file or directory: ./authorised_ping_sweep.sh

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./authorised_ping_sweep.sh 203.0.113
Scope rejected. Authorised prefix: 192.168.56

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./authorised_ping_sweep.sh 192.168.56
Authorised ICMP sweep of 192.168.56.0/24
Saved: live_hosts_20261002_110726.txt

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ cat live_hosts_20261002_110726.txt
Authorised ICMP sweep of 192.168.56.0/24
```

RESPONSE:

In this Lab I learned about input validation, the script validates the input and compare it to the condition, if met, it will authorize and save it in a file called live_hosts_timestamp. But if it authenticates the input and found out it didn't meet the condition it will prompt reject. And it is used on a specific network in this Lab. The network is "192.168.56". the script determine the prefix based on the network bits.

TASK 4 REPORT

EVIDENCE:

```
Session Actions Edit View Help

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ nano service_check.py

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ chmod +x service_check.py

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ python3 -m py_compile service_check.py

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./service_check.py 8.8.8.8
Scope rejected. Authorised network: 192.168.56.0/24

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$ ./service_check.py 192.168.56.10
CORE102 service check for 192.168.56.10
Started: 2026-10-02T11:12:45
TCP 22: CLOSED OR FILTERED
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED

(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab5]
$
```

RESPONSE:

In this Lab, I made a Python program that checks whether certain network services are working on a computer. It checks specific ports, such as 22, 80, and 8080. The program also makes sure that I only check the authorised computer

TASK 5 REPORT

EVIDENCE:

```
(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab3]
$ nano pcap_summary.py
(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab3]
$ chmod +x pcap_summary.py
(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab3]
$ ./pcap_summary.py core102_icmp.pcap | tee pcap_summary.txt
File: core102_icmp.pcap
Packets: 10
Protocols:
  ICMP      10
Top endpoints:
  192.168.149.128 10
  192.168.149.2   8
  192.168.149.254 2
Top TCP ports:
(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab3]
$ ./pcap_summary.py missing.pcap
Provide an existing .pcap or .pcapng file
(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab3]
$ ./pcap_summary.py core102_http.pcap | tee pcap_summary.txt
Provide an existing .pcap or .pcapng file
(kali@kali)-[~/Documents/ICDFA_LABS/core102_lab3]
$
```

RESPONSE:

In this Lab, I used Python and Scapy to examine a network capture file that was already recorded. The program counted the packets and showed the types of traffic, such as TCP, UDP, and ICMP. It also showed the computers involved and the TCP ports being used. I learned that both source and destination ports are counted because network communication happens between two sides. I also learned that looking at a port number alone cannot always tell us exactly which application or protocol is being used.

TASK 6 REPORT

RESPONSE:

Control	Command or evidence	Result	Conclusion
Address and route	Control	Command or evidence	Result
Firewall policy	Address and route	ip -br addr and ip route on both VMs	The private network was configured and the addresses and routes were recorded.
SSH key authentication	Firewall policy	sudo ufw status verbose	UFW was checked to confirm the default-deny incoming policy and SSH access from the private subnet.
Listening services	SSH key authentication	SSH connection from the client using the configured key	Key-based SSH login was tested from the authorised client to the server.
Host discovery	Listening services	ss -tulpn and network_inventor y.sh	The listening services and their ports were recorded.
PCAP summary	Host discovery	./authorised_ping _sweep.sh	The authorised private network was scanned for responsive hosts.
Cleanup	PCAP summary	SSH capture + ./pcap_summary.p y <PCAP>	The authorised SSH traffic was captured, stopped, and analysed offline.

TASK 7 REPORT

RESPONSE:

I reviewed the scripts using the checklist provided in the lab manual. I checked the interpreter lines, syntax and input validation, as well as the network scope and timeouts. I also checked that the output identifies the target and that sensitive information such as passwords, private keys and tokens was not included. The review helped me make sure that the scripts are safe, controlled and suitable for the authorised laboratory environment.